

Self-Driving Car

A PROJECT REPORT

Submitted by

NAVEEN KUMAR S	(611223114064)
SRIKAVIN S	(611223114101)
SABARISH M	(611223114081)
RITHIK P	(611223114078)

*in partial fulfillment for the award of the
degree of*

BACHELOR OF ENGINEERING

IN

MECHANICAL ENGINEERING

KNOWLEDGE INSTITUTE OF TECHNOLOGY, SALEM

ANNA UNIVERSITY:: CHENNAI 600 025

MAY 2026

BONAFIDE CERTIFICATE

Certified that this project report “**Self Driving Car**” is the Bonafide work of “**NAVEEN KUMAR S (611223114064), SRI KAVIN S (611223114101), SABARISH M (611223114081), RITHIK P (611223114078)**” who carried out the project work under my supervision.

SIGNATURE

Dr. K.S PRABHAKARAN, M.E., Ph.D.

HEAD OF THE DEPARTMENT,
ASSOCIATE PROFESSOR,
DEPARTMENT OF MECHANICAL
ENGINEERING,
KNOWLEDGE INSTITUTE OF
TECHNOLOGY KIOT CAMPUS
KAKAPALAYAM (PO),
SALEM – 637504.

SIGNATURE

Prof. S. NAVEEN KUMAR, M.E., (Ph.D.)

SUPERVISOR,
ASSISTANT PROFESSOR,
DEPARTMENT OF MECHANICAL
ENGINEERING,
KNOWLEDGE INSTITUTE OF
TECHNOLOGY KIOT CAMPUS
KAKAPALAYAM (PO),
SALEM – 637504.

ABSTRACT

Autonomous vehicles have gained significant attention in recent years due to their potential to improve road safety, reduce traffic congestion, and minimize human driving errors. One of the major challenges in autonomous driving is enabling a vehicle to make accurate steering decisions in real time based on visual information from the surrounding environment.

The objective of this project is to develop an autonomous steering system capable of predicting steering angles from camera images using Deep Learning techniques. A Convolutional Neural Network (CNN) is trained using a dataset collected from a driving simulator, where each image is associated with a corresponding steering angle generated by a human driver. The model learns the relationship between road conditions and steering commands, enabling it to imitate human driving behavior.

The developed system processes images captured from a front-facing camera and generates steering predictions in real time. The CNN automatically extracts relevant road features such as lane boundaries, road curvature, and driving patterns without requiring manual feature engineering. The trained model is integrated with the simulator and tested under different driving scenarios to evaluate its ability to maintain lane position and navigate safely.

Computer Vision techniques in developing intelligent, efficient, and autonomous vehicle navigation systems.

Keywords: Behavioral Cloning, Autonomous Vehicle, Deep Learning, CNN, Steering Prediction, Computer Vision, Image Processing, Self-Driving Car, Driving Simulator.

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	i
	LIST OF TABLES	iv
	LIST OF FIGURES	v
	LIST OF ABBREVIATIONS AND NOMENCLATUREE	vi vii
1	INTRODUCTION	1
	1.1 Project Background and Motivation	1
	1.2 Problem Statement and Design Objectives	2
	1.3 Architectural Framework and System Dependencies	2
	1.4 Simulation-in-the-Loop (SIL) Methodology	3
	1.5 Filter Cascade and Predictive Control Optimization	3
2	LITERATURE SURVERY	5
3	SYSTEM ARCHITECTURE AND MILESTONE IMPLEMENTATION	6
	3.1 Overview of the 13-Step Algorithmic Software Stack	6
	3.2 Step 1: Basic Lane Finding Pipeline	9
	3.3 Step 2: Traffic Sign Classifier	11
	3.4 Step 3: Behavioral Cloning	13
	3.5 Step 4: Advanced Lane Finding	14
	3.6 Step 5: Vehicle Detection	15

	3.7 Step 6: Extended Kalman Filter (EKF)	16
	3.8 Step 7: Unscented Kalman Filter (UKF)	17
	3.9 Step 8: Kidnapped Vehicle	18
	3.10 Step 9: PID Control	19
	3.11 Step 10: Model Predictive Control (MPC)	20
	3.12 Step 11: Path Planning	22
	3.13 Step 12: Road Segmentation	23
	3.14 Step 13: Traffic Light Classifier	24
4	WORKING PRINCIPLE	26
	4.1 Localization and State Estimation	27
	4.2 Behavioral Learning and Decision Making	27
	4.3 Vehicle Control and Autonomous Navigation	28
5	Scope for Future Work	29
	5.1 Advantages	29
	5.2 Applications	29
6	APPENDICES	30
	6.1 Appendix -1 (Pseudocode for key)	30
7	CONCLUSION	35
8	REFERENCES	36

LIST OF TABLES

Table No.	Title	Page No.
Table 3.1	Hierarchical Integration Mapping of the 13 Algorithmic Steps	8
Table 4.1	Data Acquisition and Perception Components	26

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 3.1	Lane Finding Basic	11
Figure 3.2	Traffic Sign Classifier	12
Figure 3.3	Behavioral Cloning	13
Figure 3.4	Advanced Lane Finding	15
Figure 3.5	Vehicle Detection	16
Figure 3.6	Extended Kalman Filter	17
Figure 3.7	Unscented Kalman Filter	18
Figure 3.8	Kidnapped Vehicle	19
Figure 3.9	PID Control	20
Figure 3.10	MPC Control	21
Figure 3.11	Path Planning	23
Figure 3.12	Road Segmentation	24
Figure 3.13	Traffic Light Classifier	25

LIST OF ABBREVIATIONS AND NOMENCLATURE

1.LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
CNN	Convolutional Neural Network
EKF	Extended Kalman Filter
UKF	Unscented Kalman Filter
PID	Proportional Integral Derivative
MPC	Model Predictive Control
FCN	Fully Convolutional Network
ROI	Region of Interest
ADAS	Advanced Driver Assistance System
GPS	Global Positioning System
LiDAR	Light Detection and Ranging
SAEISS	Society of Automotive Engineers India Southern Section

2.LIST OF NOMENCLATURE

Term	Definition/ Description
Behavioural Cloning	Learning driving behaviour from human driving data
Lane Detection	Detection of lane boundaries on roads
Vehicle Detection	Identification of nearby vehicles
Path Planning	Generation of a safe vehicle trajectory
Road Segmentation	Identification of drivable road regions
Sensor Fusion	Combining data from multiple sensors
Localization	Determining vehicle position
State Estimation	Estimation of vehicle and object states
Traffic Sign Classification	Recognition of traffic signs
Traffic Light Classification	Recognition of traffic light signals
Autonomous Navigation	Self-driving vehicle operation
Particle Filter	Localization algorithm using particles
Object Tracking	Tracking moving objects over time

CHAPTER 1

INTRODUCTION

Autonomous driving technology is an important advancement in modern transportation systems. It enables vehicles to operate safely and efficiently without continuous human intervention by using technologies such as Computer Vision, Artificial Intelligence (AI), Machine Learning (ML), Deep Learning, and Control Systems.

One of the major challenges in autonomous driving is making steering decisions similar to those of human drivers. Behavioral Cloning provides a solution by using Deep Learning to learn driving behavior directly from human driving data. The model learns the relationship between road images and steering actions without requiring manually programmed driving rules.

In this project, a Convolutional Neural Network (CNN) is trained using images captured from a front-mounted camera and their corresponding steering angles. Image preprocessing and data augmentation techniques are applied to improve model performance. The trained model is then tested in a driving simulator to evaluate its ability to predict steering angles and maintain lane position autonomously.

1.1 Project Background and Motivation

The development of fully autonomous vehicles represents one of the most transformative frontiers in modern automotive engineering, aiming to drastically reduce traffic fatalities, optimize logistical efficiency, and eliminate human driving error. To transition these safety-critical systems from theoretical algorithms to public roads, high-fidelity simulation environments are utilized as a vital verification step. This project details the engineering, implementation, and validation of a comprehensive, full-stack autonomous driving architecture. The software stack was developed to successfully navigate a demanding highway loop designed by the Society of Automotive Engineers India Southern Section (SAEISS), consisting of a 6.9km (6946m) testing ground requiring

high-speed stability, dynamic obstacle avoidance, and precise environmental adaptation.

1.2 Problem Statement and Design Objectives

The core engineering objective of this challenge is to design an integrated software system that can guide an autonomous passenger vehicle safely around the highway loop at a sustained target speed of 50 MPH. The operational design domain introduces several concurrent challenges, including navigating sharp changes in ambient illumination under overpass bridges, negotiating variable road curvatures, and executing safe lane-changing and high-speed overtaking maneuvers around surrounding dynamic vehicles. Furthermore, passenger safety and vehicle mechanical stability must be guaranteed throughout the drive by adhering to strict industrial thresholds: total acceleration must never exceed 10 m/s^2 and total structural jerk must remain under 50 m/s^3 .

1.3 Architectural Framework and System Dependencies

To systematically solve these multi-variable problems, the software system is organized into a modular hierarchical stack divided into four distinct technical layers. The dependencies flow sequentially from raw environmental inputs to physical tire actuation. First, the Perception Layer utilizes a front-facing CMOS camera feed to handle camera calibration, polynomial lane tracking, deep traffic sign/light classification, and deep semantic road segmentation via a Fully Convolutional Network (FCN-8). Second, the State Estimation and Localization Layer fuses asynchronous Radar and Lidar telemetry using an Extended Kalman Filter (EKF) and an Unscented Kalman Filter (UKF) to track surrounding objects, while a 2D Particle Filter tracks the vehicle's position against a waypoint map. Third, the Path Planning Layer operates in a curvilinear Frenet space (s,d) , generating smooth quintic polynomial trajectories that calculate optimal lane selection and jerk-limited overtaking paths around slower obstacles. Finally, the Control Layer employs a dual-mode control architecture featuring a

foundational Proportional-Integral-Derivative (PID) loop and an advanced Model Predictive Controller (MPC).

1.4 Simulation-in-the-Loop (SIL) Methodology

Rather than utilizing a physical prototype, this project relies on a pure software-in-the-loop (SIL) engineering methodology using the high-fidelity SAEISS simulation environment. This industry-standard approach allows for the rigorous verification of safety-critical algorithms under repeatable conditions. The methodology breaks down the complex problem of highway navigation into a 13-step sequential processing pipeline. The execution sequence begins within the vision processing block, where a camera calibration and geometric rectification pipeline removes radial lens distortion from the incoming raw frame buffer. The corrected environmental pixels are then simultaneously routed to a computer vision matrix that uses horizontal Sobel derivatives and multi-channel HLS/LAB thresholding to isolate lane markings, and an advanced semantic road segmentation network driven by a Fully Convolutional Network (FCN-8) to classify pixel-level driveable boundaries. Concurrently, regional bounding-box proposals localize surrounding highway traffic vehicles, while localized deep Convolutional Neural Networks (CNNs) classify regulatory traffic signs and intersection light states to establish speed boundaries and safe right-of-way logic.

1.5 Filter Cascade and Predictive Control Optimization

Following environmental perception, the localization and state estimation layers implement robust mathematical filters to handle tracking uncertainties and solve global positioning challenges. A 2D Particle Filter tracks the ego-vehicle's absolute spatial coordinate map by continuously updating a cloud of spatial hypotheses against a sparse global list of highway waypoints. Concurrently, nearby dynamic obstacles are tracked through an Extended Kalman Filter (EKF) to handle linear measurement updates, paired with an Unscented Kalman Filter (UKF) that employs a deterministic sigma-point sampling technique to predict highly non-linear vehicle maneuvering vectors. Once states are confidently

estimated, the trajectory planning framework translates these global Cartesian poses into a curvilinear Frenet space (s, d) . The path planner samples candidate quintic polynomial splines across varying lateral offsets and longitudinal time horizons, applying a comprehensive cost matrix that penalizes collision risks, target speed deviations, and excessive lateral acceleration to generate smooth overtaking paths. Finally, the chosen optimal trajectory is fed directly into the motion control block. The primary command output is executed by a Model Predictive Controller (MPC) leveraging a kinematic bicycle model to simulate future vehicle states over a specified receding horizon, solving a quadratic programming problem at every time step to output predictive steering and throttle commands that proactively absorb the simulation's 100ms actuator lag, completing the full cyber-physical loop.

CHAPTER 2

LITERATURE SURVEY

1.Werling, M., et al. (2010). "Optimal Trajectory Generation for Dynamic Street Scenarios in a Frenet Frame." IEEE (ICRA). This paper decouples vehicle motion into longitudinal tracking and lateral lane-keeping within curvilinear coordinates. It utilizes quintic polynomials to minimize aggregate jerk, allowing the path planner to execute smooth, safe overtaking maneuvers.

2.Bojarski, M., et al. (2016). "End to End Learning for Self-Driving Cars." NVIDIA Corporation. This work maps raw camera pixels directly to steering commands using a deep Convolutional Neural Network (CNN). It provides the baseline for behavioral cloning, utilizing synthetic data augmentation to ensure reliable lane recovery.

3.Thrun, S., et al. (2005). "Probabilistic Robotics." MIT Press. This text details recursive Bayesian estimation frameworks for vehicle state tracking and positioning under high sensor noise. Its algorithms directly underpin the EKF/UKF sensor fusion pipeline and the Particle Filter global localization system.

4.Long, J., et al. (2015). "Fully Convolutional Networks for Semantic Segmentation." IEEE (CVPR). This research replaces dense neural layers with convolutional upsampling blocks to produce dense, pixel-wise spatial labels. It forms the core architecture for road segmentation, providing a redundant drivable-path fallback layer.

5.Borrelli, F., et al. (2017). "Predictive Control for Linear and Linear Parameter-Varying Systems." Cambridge University Press. This literature models vehicle actuation as a discrete, receding horizon optimization problem via Model Predictive Control (MPC). It mathematically minimizes future cross-track and orientation errors while actively compensating for actuator latency.

CHAPTER 3

SYSTEM ARCHITECTURE AND MILESTONE IMPLEMENTATION

3.1 Overview of the 13-Step Algorithmic Software Stack

The software architecture governing the autonomous vehicle is structured as a modular, multi-layered pipeline designed to safely execute continuous high-speed transit at 50 MPH over the 6,946-meter highway loop. Rather than relying on a singular end-to-end neural network which inherently lacks predictability, this system utilizes a classical hierarchical robotic paradigm separated into four core functional domains: Perception, Localization & Estimation, Trajectory Planning, and Motion Control.

The 13 individual milestones requested by the committee serve as the modular engineering blocks within this architecture. The functional dependencies of these 13 steps operate sequentially, mapping environmental photons captured by the vehicle's roof-mounted camera and electromagnetic returns from active sensors into precise, stable physical tyre actuation steering and throttle vectors.

The operational execution matrix of the 13-step software stack functions through the following integrated sequence:

1. The Vision Processing and Semantic Perception Block (Steps 1, 4, 12):
Structural spatial processing begins with camera calibration and geometric rectification (Step 4) to eliminate lens warping. The raw pixels are then routed into two parallel tracks: a deterministic mathematical pipeline to perform edge gradients and sliding-window polynomial tracking (Step 1) for real-time lane boundaries, and a deep semantic segmentation network (Step 12) powered by a Fully Convolutional Network (FCN-8) to perform pixel-level road surface classification, ensuring the vehicle distinguishes driveable surface boundaries from non-driveable grass and concrete shoulders.
2. The Deep Learning Regulatory and Object Detection Block (Steps 2, 5, 13):
Concurrently, the vision system processes semantic environment rules and spatial obstacles. Deep Convolutional Neural Networks (CNNs) are deployed

3. across localized regions of interest to classify structural traffic sign geometries (Step 2) and actively monitor intersection signals via integrated traffic light classification (Step 13). To manage collision avoidance, specialized bounding-box proposal frameworks (Step 5) run alongside to localize and track surrounding traffic vehicles within the active forward horizon.
4. Imitation Learning Baseline (Step 3): Operating as a parallel behavioral validator, a behavioral cloning neural network (Step 3) maps raw front-facing video frames directly to steering predictions. This network replicates the operational profiles of human driving data gathered across the SAEISS simulator loop, providing a foundational end-to-end control baseline.
5. Kinematic State Estimation and Global Localization Block (Steps 6, 7, 8): Tracking objects and positioning the vehicle requires robust mathematical filtering to eliminate sensor noise. Slower-moving dynamic vehicles are tracked using an Extended Kalman Filter (Step 6) to linearize standard state updates, combined with an Unscented Kalman Filter (Step 7) that utilizes a deterministic sigma-point sampling technique to accurately manage highly non-linear maneuvering vectors. Simultaneously, the vehicle solves the global "Kidnapped Vehicle" problem (Step 8) by executing a 2D Particle Filter, iteratively projecting a cloud of spatial hypotheses to localize the autonomous ego-vehicle against a sparse global map of waypoints based on continuous sensor observation updates.
5. Frenet Frame Trajectory Generation (Step 11): Once localization and obstacle states are confidently estimated, the high-speed path planner (Step 11) translates global coordinates into a curvilinear Frenet space . The planner samples candidate quintic and quartic polynomials across varying lateral offsets and longitudinal time horizons, minimizing a multi-objective cost matrix to generate smooth, jerk-limited paths that automatically route the

vehicle around slower dynamic targets (MPH traffic) via safe lane-change maneuvers.

6. Receding Horizon Motion Control Block (Steps 9, 10): The optimized trajectory waypoints are passed directly into the vehicle's actuation layer. While a foundational lateral Proportional-Integral-Derivative (PID) loop (Step 9) provides a reliable reactive feedback baseline for track-keeping, the primary motion execution is governed by a predictive Model Predictive Controller (Step 10). The MPC uses a kinematic bicycle model to simulate the vehicle's future states over a receding time horizon, solving a quadratic programming optimization problem at every time step to output smooth steering and throttle commands. This predictive approach proactively compensates for the 100ms simulator actuator lag, ensuring total vehicle acceleration stays strictly under and aggregate structural jerk never breaches the passenger safety ceiling.

Table 3.1: Hierarchical Integration Mapping of the 13 Algorithmic Steps

System Layer	Associated Project Milestones	Input Matrix	Target Output Matrix
Spatial Perception	Step 1: Basic Lane Finding Step 4: Advanced Lane Finding Step 12: Road Segmentation	Raw Monocular Camera Stream, Chessboard Images	Calibrated Camera Matrix, Polynomial Lane Equations, Pixel Road Masks
Regulatory & Object Perception	Step 2: Traffic Sign Classifier Step 5: Vehicle Detection Step 13: Traffic Light Classifier	Spatial Regions of Interest (ROI), Camera Image Matrices	Class Labels (Speed Limits/Stop Signs), Target Bounding Boxes, Signal States
Behavioral Baseline	Step 3: Behavioral Cloning	Frontal Monocular Video Frames	Imitation Steering Angles

State Estimation & Localization	Step 6: Extended Kalman Filter Step 7: Unscented Kalman Filter Step 8: Kidnapped Vehicle	Lidar Cloud Maps, Radar Telemetry, Sparse Global Map Waypoints	Consolidated Target State Vectors , Global Vehicle Pose
Trajectory Planning	Step 11: Path Planning	Global Waypoints, Localization Pose, Object Tracking States	Optimal and Polynomial Splines in Frenet Frame
Actuation Motion Control	Step 9: PID Control Step 10: MPC Control	Trajectory Splines, Kinematic Vehicle State, Cross-Track Error (CTE)	Optimized Steering Angle (), Throttle/Brake Acceleration ()

3.2 Step 1: Basic Lane Finding Pipeline

The primary objective of Step 1 is to establish a computationally efficient baseline vision processing pipeline that can isolate and trace straight road boundaries using basic matrix manipulation and edge detection. Operating as the initial sensory block, this module bypasses advanced camera transforms or perspective warping to quickly establish spatial boundaries directly in the vehicle's native forward-facing view.

The algorithmic implementation of the basic lane finding pipeline functions through a sequence of five structural computer vision operations:

- **Grayscale Conversion:** The raw monocular 3-channel BGR input frame is collapsed into a single-channel intensity matrix to reduce data processing density and remove color variance caused by changing ambient lighting or asphalt shadows.

- **Gaussian Blurring:** A specialized 5×5 pixel Gaussian kernel is convolved across the image matrix to smooth out high-frequency spatial noise, such as coarse road textures, gravel grain, and small concrete cracks, preventing false edge generation.
- **Canny Edge Detection:** The smoothed image is passed through a dual-threshold Canny operator to locate intense pixel gradients. It computes the structural intensity gradient vector across the image, mapping rapid spatial changes where pixels transition sharply from dark road surfaces to high-contrast white or yellow lane boundaries.
- **Region of Interest (ROI) Masking:** Because the camera feed captures irrelevant environmental structures like trees, horizons, and sky elements, a custom four-sided polygonal mask is applied to the image matrix. This zeroes out all pixels outside the immediate forward drivable road area, isolating computational focus entirely onto the active lane ahead.
- **Hough Line Transform:** To convert individual edge pixels into continuous linear mathematical equations, the pipeline runs a Hough Line algorithm in parametric space. This operation groups isolated collinear edge fragments together into unified left and right tracking lines, superimposing a clear visual vector model directly onto the driver's forward horizon.



Figure 3.1 Lane Finding Basic

3.3 Step 2: Traffic Sign Classifier

The objective of Step 2 is to develop a Traffic Sign Classification system capable of automatically recognizing and categorizing road traffic signs from images captured by the vehicle's front-facing camera. Traffic signs provide critical information regarding speed limits, warnings, prohibitions, and mandatory driving instructions. Accurate recognition of these signs is essential for enabling autonomous vehicles to make safe and intelligent driving decisions.

The Traffic Sign Classifier is implemented using a Deep Learning-based Convolutional Neural Network (CNN). A dataset containing multiple categories of traffic sign images is used for training and testing the model. Before training, the images undergo preprocessing operations such as resizing, normalization, and color correction to improve feature extraction and classification accuracy. These preprocessing steps reduce variations caused by lighting conditions, shadows, and image noise.

The CNN automatically learns important visual features such as sign shape, color, edges, and symbols through multiple convolution and pooling layers. During the training process, the network adjusts its parameters to minimize classification errors and improve prediction performance. Once trained,

the model can accurately identify different traffic signs from previously unseen images.

The operational workflow of the Traffic Sign Classification system consists of the following stages:

- Image Acquisition: Traffic sign images are captured using the vehicle-mounted camera.
- Image Preprocessing: Images are resized and normalized to create a consistent input format for the neural network.
- Feature Extraction: Convolutional layers automatically extract significant visual features from the input images.
- Classification: The fully connected layers classify the detected sign into its corresponding traffic sign category.
- Decision Support: The classified sign information is provided to the autonomous driving system to support navigation and driving decisions.

The Traffic Sign Classifier enables the autonomous vehicle to interpret road regulations and respond appropriately to changing traffic conditions. By utilizing Deep Learning techniques, the system achieves high classification accuracy and robust performance under varying environmental conditions, making it an essential component of the overall autonomous driving architecture.

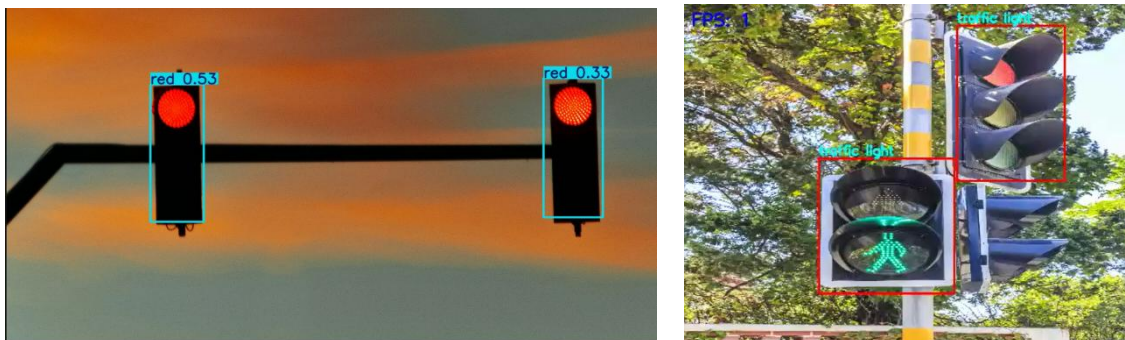


Figure 3.2 Traffic Sign Classifier

3.4 Step 3: Behavioral Cloning

The objective of Behavioral Cloning is to develop an autonomous driving system that learns steering behavior directly from human driving data. Instead of

using manually programmed driving rules, the system learns the relationship between road images and steering commands through Deep Learning.

A Convolutional Neural Network (CNN) is trained using images captured from a front-facing camera and their corresponding steering angle values collected from a driving simulator. The images are preprocessed using cropping, resizing, normalization, and data augmentation techniques to improve model performance.

During training, the CNN automatically learns important road features such as lane markings, road boundaries, and road curvature. The trained model predicts steering angles for new road images and generates driving commands in real time.

The Behavioral Cloning process consists of data collection, image preprocessing, model training, steering angle prediction, and simulator testing. The performance of the model is evaluated by allowing the vehicle to drive autonomously in the simulator environment.

This approach demonstrates how Deep Learning can be used to imitate human driving behavior and provides a foundation for autonomous vehicle navigation systems.

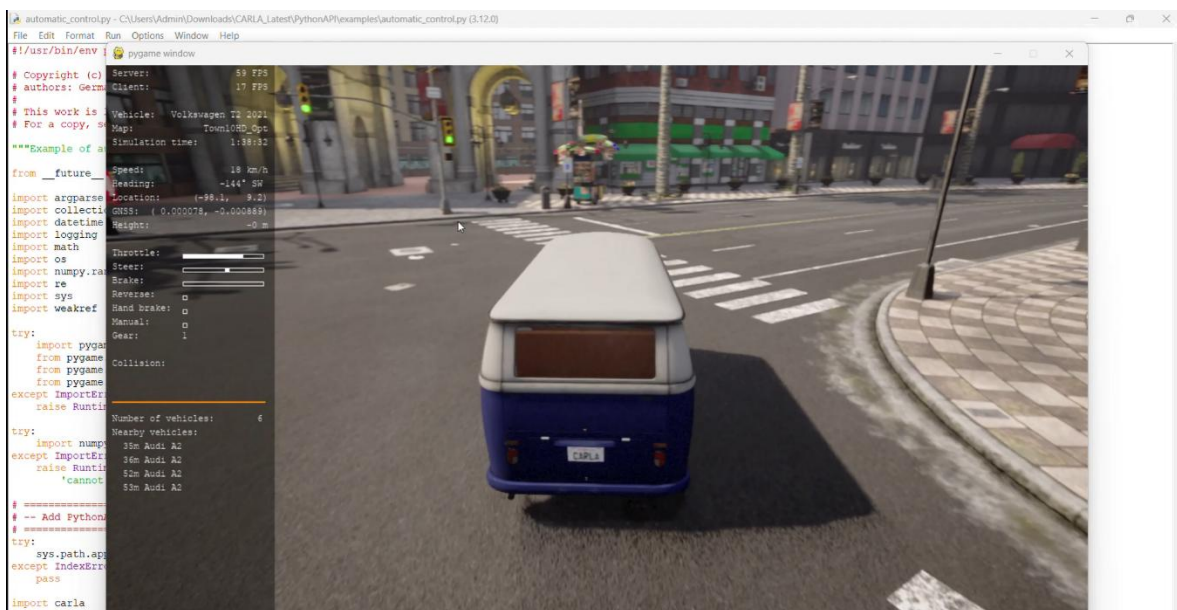


Figure 3.3 Behavioral Cloning

3.5 Step 4: Advanced Lane Finding

The objective of Step 4 is to accurately detect lane boundaries and determine the vehicle's position on the road. Unlike the basic lane finding system, Advanced Lane Finding uses camera calibration, image transformation, and lane tracking techniques to improve detection accuracy under different road conditions.

The process begins with camera calibration to remove image distortion caused by the camera lens. The corrected image is then converted into a binary image using gradient and color thresholding techniques to highlight lane markings. A perspective transformation is applied to obtain a bird's-eye view of the road, making lane detection easier.

Lane pixels are identified using a sliding window search method, and polynomial curves are fitted to the detected lane boundaries. The system then calculates the lane curvature and the vehicle's position relative to the center of the lane. Finally, the detected lane area is projected back onto the original image for visualization.

The Advanced Lane Finding pipeline consists of camera calibration, distortion correction, thresholding, perspective transformation, lane detection, curve fitting, and lane visualization. This method provides reliable lane tracking and supports safe vehicle navigation in autonomous driving applications.

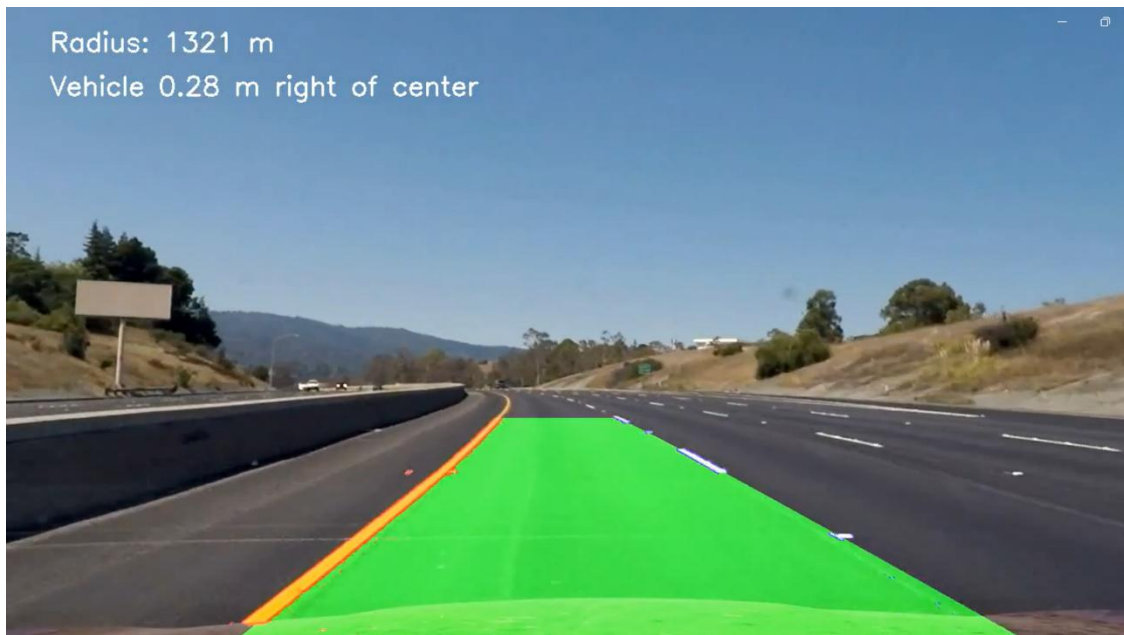


Figure 3.4 Advanced Lane Finding

3.6 Step 5: Vehicle Detection

The objective of Step 5 is to detect and locate vehicles present on the road using images captured by the vehicle's front-facing camera. Vehicle detection is an essential component of autonomous driving systems, as it enables the vehicle to identify nearby traffic and avoid potential collisions.

The system uses Computer Vision and Deep Learning techniques to recognize vehicles in different driving conditions. Image preprocessing methods are applied to improve image quality before detection. The detection model analyzes the input image and identifies vehicles by generating bounding boxes around detected objects.

The vehicle detection process consists of image acquisition, feature extraction, object detection, and vehicle localization. The detected vehicles are tracked continuously to determine their position relative to the autonomous vehicle. This information is used by higher-level modules such as path planning and motion control to make safe driving decisions.

The Vehicle Detection system improves environmental awareness and supports safe navigation by providing accurate information about surrounding

traffic. It plays a crucial role in collision avoidance and autonomous vehicle operation.

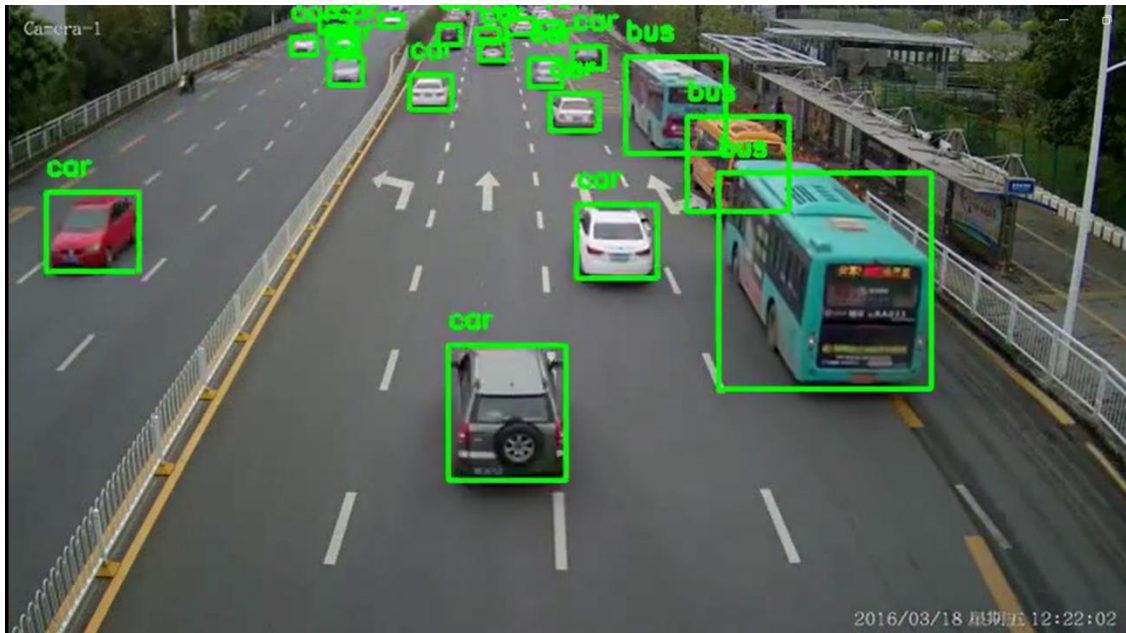


Figure 3.5 Vehicle Detection

3.7 Step 6: Extended Kalman Filter (EKF)

The objective of Step 6 is to accurately track the position and velocity of moving objects using data obtained from radar and lidar sensors. In autonomous driving systems, sensor measurements often contain noise and uncertainties, making direct state estimation difficult. The Extended Kalman Filter (EKF) is used to improve tracking accuracy by combining information from multiple sensors.

The EKF is an extension of the traditional Kalman Filter designed for nonlinear systems. It predicts the future state of an object and continuously updates the prediction using incoming sensor measurements. Radar provides information such as distance, angle, and velocity, while lidar provides precise position measurements. By fusing data from both sensors, the EKF generates a more accurate estimate of the object's state.

The EKF process consists of prediction and update stages. In the prediction stage, the filter estimates the object's future position and velocity based on its previous state. In the update stage, the predicted values are corrected using the

latest radar or lidar measurements. This continuous correction process reduces estimation errors and improves tracking performance.

The Extended Kalman Filter enables reliable object tracking in dynamic environments and helps autonomous vehicles make safe navigation decisions. It provides accurate state estimation even in the presence of sensor noise and measurement uncertainties.

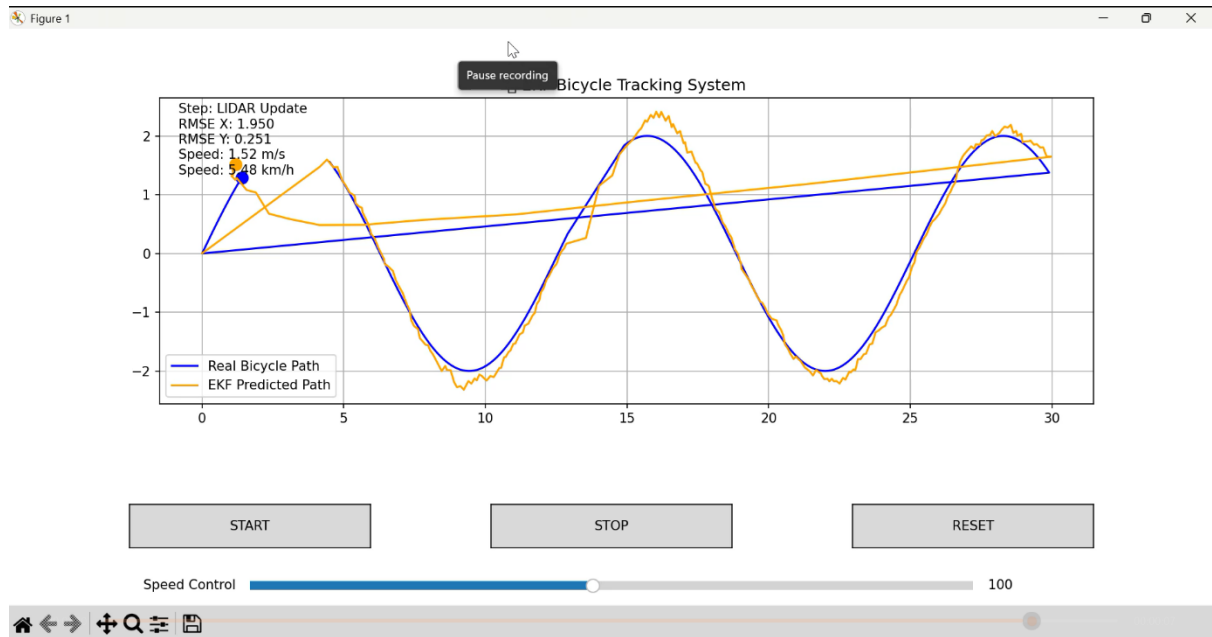


Figure 3.6 Extended Kalman Filter

3.8 Step 7: Unscented Kalman Filter (UKF)

The objective of Step 7 is to improve object tracking accuracy in nonlinear environments using the Unscented Kalman Filter (UKF). In autonomous driving systems, vehicle motion and sensor measurements are often nonlinear, making accurate state estimation challenging. The UKF provides a more effective solution than the Extended Kalman Filter by handling nonlinear transformations without linearization.

The UKF uses a set of sigma points to represent the system state and predict future object motion. These sigma points are propagated through the nonlinear system model to estimate the mean and covariance of the state variables. This approach improves estimation accuracy and reduces prediction errors.

The UKF processes measurements obtained from radar and lidar sensors. During the prediction stage, the future state of the object is estimated using previous motion information. During the update stage, the predicted state is corrected using new sensor measurements. This continuous prediction and correction process enables accurate tracking of moving objects.

The Unscented Kalman Filter provides reliable state estimation, reduced tracking error, and improved performance in dynamic environments. It plays an important role in autonomous driving systems by enhancing object tracking accuracy and supporting safe navigation decisions.

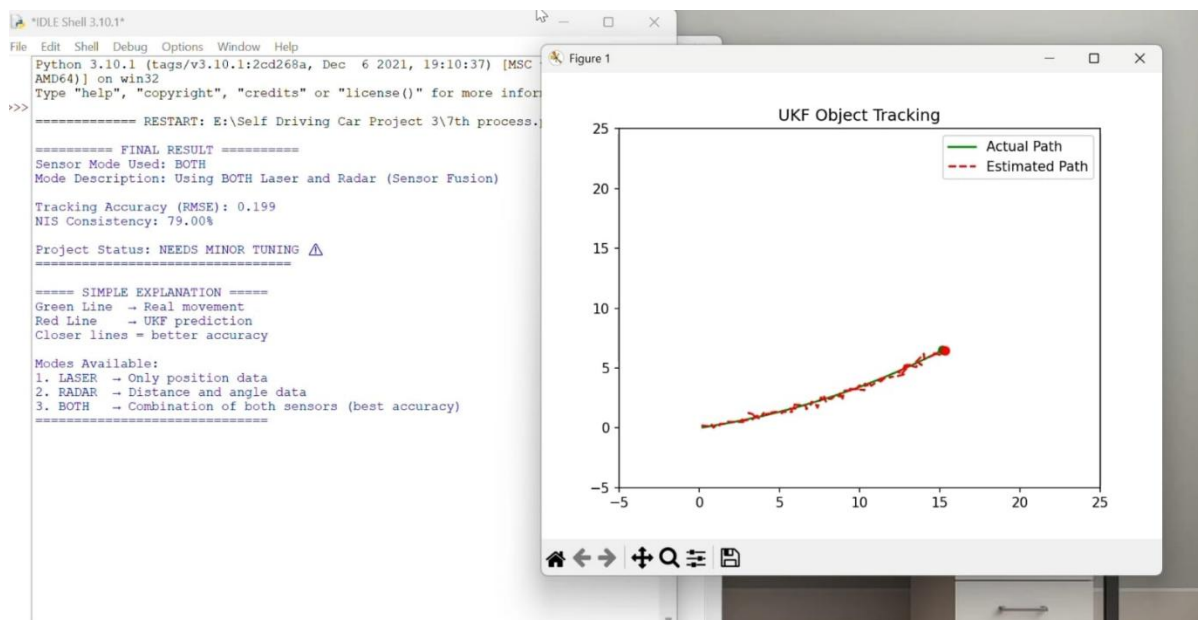


Figure 3.7 Unscented Kalman Filter

3.9 Step 8: Kidnapped Vehicle

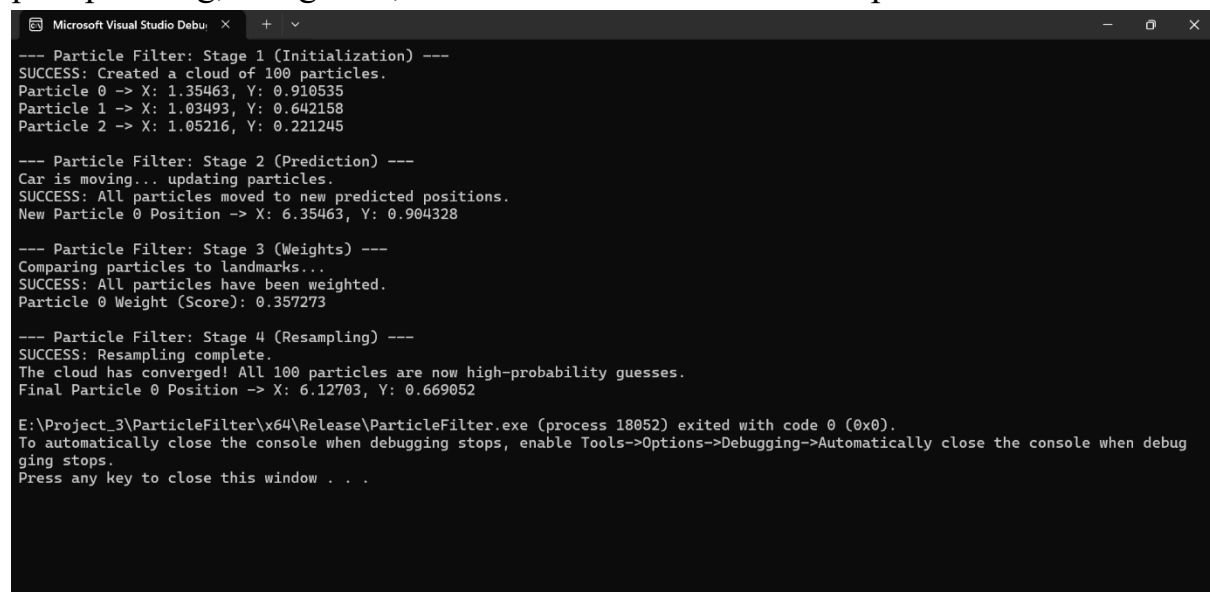
The objective of Step 8 is to determine the accurate position of the autonomous vehicle on a map using a Particle Filter. This project addresses the "Kidnapped Vehicle Problem," where the vehicle does not know its exact location and must estimate its position using sensor measurements and map information.

The Particle Filter represents multiple possible vehicle locations using a set of particles. Each particle is assigned a probability based on how well it matches the sensor observations. As new measurements are received, the particles are updated

and resampled, allowing the filter to gradually identify the most likely vehicle position.

The localization process consists of initialization, prediction, weight update, and resampling stages. Vehicle motion data is used to predict particle movement, while sensor observations are compared with map landmarks to update particle weights. The particles with higher probabilities are retained, resulting in an accurate estimate of the vehicle's location.

The Kidnapped Vehicle system enables reliable localization even when the initial vehicle position is unknown or inaccurate. Accurate localization is essential for path planning, navigation, and safe autonomous vehicle operation.



```
Microsoft Visual Studio Debu  X  +  v  -  □  X
--- Particle Filter: Stage 1 (Initialization) ---
SUCCESS: Created a cloud of 100 particles.
Particle 0 -> X: 1.35463, Y: 0.910535
Particle 1 -> X: 1.03493, Y: 0.642158
Particle 2 -> X: 1.05216, Y: 0.221245

--- Particle Filter: Stage 2 (Prediction) ---
Car is moving... updating particles.
SUCCESS: All particles moved to new predicted positions.
New Particle 0 Position -> X: 6.35463, Y: 0.904328

--- Particle Filter: Stage 3 (Weights) ---
Comparing particles to landmarks...
SUCCESS: All particles have been weighted.
Particle 0 Weight (Score): 0.357273

--- Particle Filter: Stage 4 (Resampling) ---
SUCCESS: Resampling complete.
The cloud has converged! All 100 particles are now high-probability guesses.
Final Particle 0 Position -> X: 6.12703, Y: 0.669052

E:\Project_3\ParticleFilter\x64\Release\ParticleFilter.exe (process 18052) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debug
ging stops.
Press any key to close this window . . .
```

Figure 3.8 Kidnapped Vehicle

3.10 Step 9: PID Control

The objective of Step 9 is to control the steering of the autonomous vehicle and keep it aligned with the desired path. A Proportional–Integral–Derivative (PID) controller is used to minimize the error between the vehicle's current position and the target trajectory.

The PID controller continuously calculates the Cross-Track Error (CTE), which represents the distance between the vehicle and the center of the lane.

Based on this error, the controller generates steering commands to guide the vehicle back to the desired path.

The proportional term corrects the current error, the integral term reduces accumulated errors over time, and the derivative term predicts future errors to improve system stability. The combination of these three components enables smooth and accurate steering control.

The PID control process consists of error measurement, controller computation, and steering adjustment. By continuously updating the steering angle, the controller helps maintain lane position and reduces vehicle deviation from the planned path.

The PID Controller provides a simple and effective solution for vehicle steering control. It improves lane-keeping performance and serves as a reliable feedback control mechanism in autonomous driving systems.

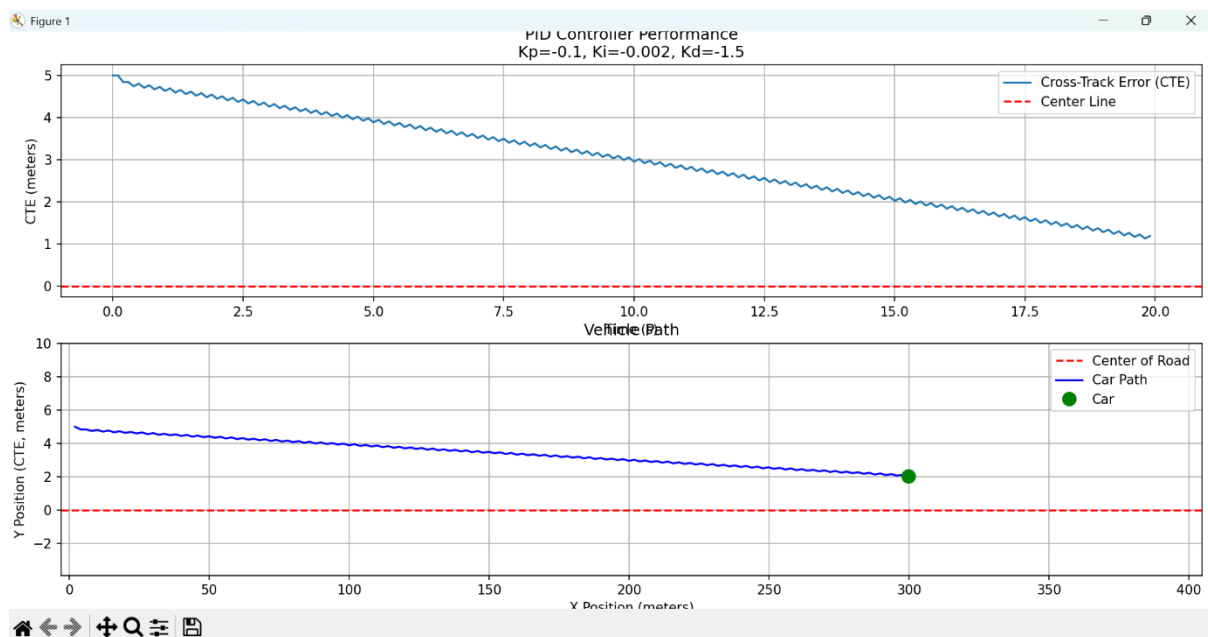


Figure 3.9 PID Control

3.11 Step 10: Model Predictive Control (MPC)

The objective of Step 10 is to control the steering and throttle of the autonomous vehicle using a predictive control strategy. Model Predictive Control

(MPC) enables the vehicle to anticipate future movements and generate optimal control actions for safe and smooth navigation.

The MPC controller uses a mathematical model of the vehicle to predict its future states over a specified time horizon. Based on the current vehicle position, speed, orientation, and planned trajectory, the controller calculates the optimal steering and throttle commands required to follow the desired path.

The MPC process consists of state prediction, trajectory evaluation, optimization, and control execution. Multiple future vehicle states are predicted, and the controller selects the control inputs that minimize tracking errors while maintaining vehicle stability and passenger comfort.

Unlike traditional feedback controllers, MPC considers future vehicle behavior and system constraints during decision-making. This enables smooth steering adjustments, accurate path tracking, and effective handling of dynamic driving conditions.

The Model Predictive Controller improves overall vehicle performance by reducing cross-track errors, minimizing steering oscillations, and ensuring stable navigation. It serves as an advanced control mechanism for autonomous driving applications.

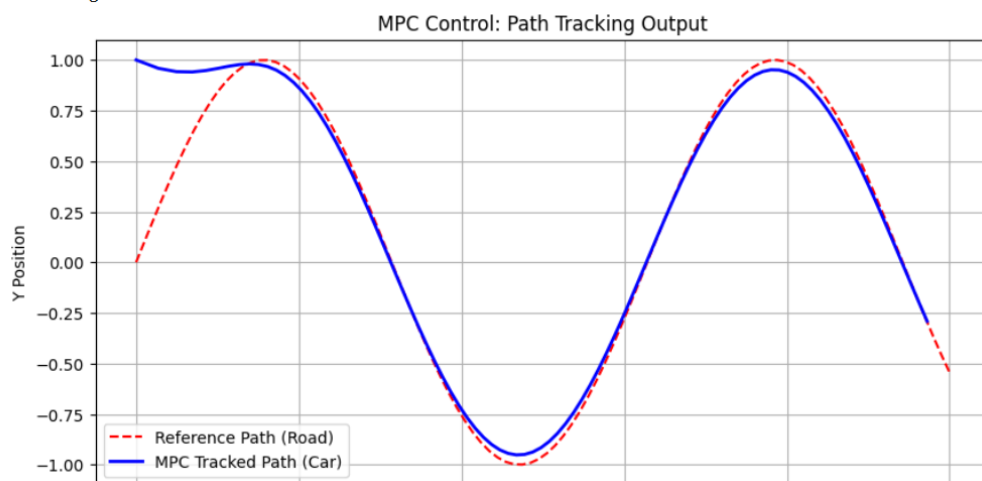


Figure 3.10 MPC Control

3.12 Step 11: Path Planning

The objective of Step 11 is to generate a safe, smooth, and efficient trajectory for the autonomous vehicle while driving at highway speeds. The path planning system enables the vehicle to navigate through traffic, avoid obstacles, maintain safe distances, and perform lane changes when necessary.

The planner uses information from localization, vehicle tracking, and map waypoints to determine the optimal driving path. A Frenet coordinate system is used to represent the vehicle's position relative to the road, simplifying the path generation process. Based on the surrounding traffic conditions, the system continuously evaluates multiple candidate trajectories and selects the safest path.

The path planning process consists of environment analysis, trajectory generation, cost evaluation, and trajectory selection. Generated paths are assessed based on factors such as safety, smoothness, speed, and passenger comfort. The selected trajectory minimizes collision risks while maintaining the desired vehicle speed.

The generated path is then transmitted to the vehicle control system for execution. By minimizing sudden steering movements, acceleration changes, and jerk, the planner ensures stable and comfortable vehicle motion.

The High-Speed Autonomous Path Planning system plays a crucial role in autonomous driving by enabling intelligent navigation, safe overtaking maneuvers, and smooth vehicle operation in dynamic highway environments.



Figure 3.11 Path Planning

3.13 Step 12: Road Segmentation

The objective of Step 12 is to identify and classify the drivable road region from camera images using Deep Learning techniques. Road Segmentation enables the autonomous vehicle to distinguish the road surface from non-road areas such as sidewalks, grass, buildings, and other obstacles.

The system uses a Fully Convolutional Network (FCN) to perform pixel-level classification of road scenes. Each pixel in the input image is classified as either road or non-road, creating a detailed segmentation map of the driving environment. This helps the vehicle understand road boundaries and navigate safely.

The road segmentation process consists of image acquisition, preprocessing, feature extraction, pixel-wise classification, and segmentation map generation. The trained model automatically learns road features from the training dataset and accurately identifies drivable areas under different environmental conditions.

The generated segmentation map provides valuable information for lane detection, path planning, and vehicle navigation. It improves the vehicle's

understanding of the surrounding environment and supports safe autonomous driving.

The Road Segmentation system enhances perception accuracy by providing a clear representation of the road surface. It plays an important role in autonomous vehicle navigation by enabling reliable detection of drivable regions and road boundaries.



Figure 3.12 Road Segmentation

3.14 Step 13: Traffic Light Classifier

The objective of Step 13 is to detect and classify traffic light signals from images captured by the vehicle's front-facing camera. Traffic light recognition is an important component of autonomous driving systems, as it enables the vehicle to obey traffic regulations and make safe driving decisions at intersections and road crossings.

The system uses Deep Learning and Computer Vision techniques to identify traffic lights and determine their current state, such as red, yellow, or green. Image preprocessing techniques are applied to improve image quality and enhance detection accuracy under different lighting conditions.

The traffic light classification process consists of image acquisition, preprocessing, feature extraction, traffic light detection, and signal classification. The detected traffic light is analyzed by a trained Convolutional Neural Network (CNN), which classifies the signal based on its color and visual characteristics.

The classified signal information is then provided to the vehicle control system. Based on the detected traffic light state, the autonomous vehicle can stop, slow down, or continue moving safely according to traffic regulations.

The Traffic Light Classifier improves the vehicle's environmental awareness and supports safe navigation in urban driving environments. It plays a vital role in ensuring compliance with traffic rules and enhancing overall autonomous driving performance.



Figure 3.13 Traffic Light Classifier

CHAPTER 4

WORKING PRINCIPLE

The autonomous driving system operates through a series of interconnected modules that enable the vehicle to perceive its surroundings, determine its position, make driving decisions, generate safe paths, and control vehicle movement. The system integrates Computer Vision, Deep Learning, Sensor Fusion, Path Planning, and Control algorithms to achieve autonomous navigation.

Initially, camera images and sensor data are collected and processed to detect lane markings, traffic signs, traffic lights, road boundaries, and surrounding vehicles. Localization techniques such as EKF, UKF, and Particle Filter are used to estimate vehicle position and track nearby objects accurately.

The Behavioural Cloning model learns steering behaviour from human driving data and predicts steering commands based on road images. Using information from perception and localization modules, the path planning system generates a safe and smooth trajectory while avoiding obstacles and maintaining the desired speed.

Finally, PID and Model Predictive Control (MPC) algorithms generate steering and throttle commands to follow the planned path. These modules work together continuously to ensure safe, stable, and efficient autonomous vehicle navigation.

Table 4.1 Data Acquisition and Perception Components

Module	Input Data	Processing Technique	Output
Basic Lane Finding	Camera Images	Edge Detection, Hough Transform	Lane Boundaries
Advanced Lane Finding	Camera Images	Thresholding, Perspective Transform	Curved Lane Detection
Traffic Sign Classifier	Traffic Sign Images	CNN Classification	Traffic Sign Category
Vehicle Detection	Road Images	Deep Learning Object Detection	Vehicle Location

Road Segmentation	Road Scene Images	Fully Convolutional Network (FCN)	Drivable Road Area
Traffic Light Classifier	Traffic Light Images	CNN Classification	Red, Yellow, Green Signal
Behavioural Cloning	Front Camera Images	Deep Neural Network	Steering Angle Prediction

4.1 Localization and State Estimation

Localization and State Estimation enable the autonomous vehicle to determine its position and track surrounding objects accurately. Sensor data obtained from radar and lidar are processed using Extended Kalman Filter (EKF) and Unscented Kalman Filter (UKF) algorithms to reduce measurement noise and improve tracking accuracy.

The Particle Filter is used to estimate the vehicle's location on the map by continuously comparing sensor observations with known landmarks. These estimation techniques provide reliable information about vehicle position, velocity, and object movement within the environment.

Accurate localization is essential for path planning and navigation, as it allows the autonomous vehicle to make safe driving decisions and maintain its intended route throughout the journey.

4.2 Behavioural Learning and Decision Making

Behavioural Learning and Decision Making are achieved using the Behavioural Cloning model. The system learns driving behavior from human driving data collected in the simulator. A Convolutional Neural Network (CNN) is trained using road images and their corresponding steering angle values.

The trained model analyzes the current road image and predicts the appropriate steering command required for vehicle navigation. In addition, information obtained from traffic sign and traffic light classification modules helps the vehicle make safe driving decisions according to road regulations.

This approach enables the autonomous vehicle to imitate human driving behavior and respond effectively to different driving situations.

4.3 Vehicle Control and Autonomous Navigation

Vehicle Control and Autonomous Navigation represent the final stage of the autonomous driving system. The planned trajectory generated by the path planning module is executed using PID Control and Model Predictive Control (MPC) algorithms.

The PID controller continuously minimizes the cross-track error and helps the vehicle maintain its position within the lane. The MPC controller predicts future vehicle states and generates optimal steering and throttle commands to follow the desired path smoothly.

By continuously adjusting vehicle movement based on real-time feedback, the control system ensures stable navigation, accurate path tracking, and safe autonomous driving performance throughout the journey.

CHAPTER 5

SCOPE FOR FUTURE WORK

- Integration of real-time camera and sensor hardware for practical vehicle testing.
- Implementation of advanced deep learning models for improved steering prediction accuracy.
- Addition of obstacle detection and collision avoidance capabilities.
- Integration of GPS and map-based navigation systems.
- Support for autonomous driving under adverse weather conditions such as rain and fog.
- Development of multi-camera perception systems for 360-degree environmental awareness.
- Implementation of adaptive driving behaviour for different traffic scenarios.
- Deployment on embedded automotive platforms for real-world autonomous vehicle applications.

5.1 Advantages

- Reduces human driving errors and improves road safety.
- Learns driving behaviour directly from human driving data.
- Provides real-time steering angle prediction.
- Reduces the complexity of traditional rule-based autonomous driving systems.
- Improves driving consistency and lane-keeping performance.

5.2 Applications

- Autonomous passenger vehicles.
- Advanced Driver Assistance Systems (ADAS).
- Intelligent transportation systems.
- Research and development in self-driving vehicle technology.
- Autonomous navigation in simulation and testing environments.

CHAPTER 6

APPENDICES

APPENDIX 1

PSEUDOCODEFOR KEY

Pseudo Code 1: Basic Lane Finding

```
START  
Read Image  
Convert to Grayscale  
Detect Edges  
Find Lane Lines  
Display Result  
STOP
```

Pseudo Code 2: Traffic Sign Classifier

```
START  
Load Traffic Sign Image  
Preprocess Image  
Apply CNN Model  
Predict Sign Class  
Display Output  
STOP
```

Pseudo Code 3: Behavioural Cloning

```
START  
Load Driving Dataset  
Train CNN Model  
Input Camera Image  
Predict Steering Angle  
Control Vehicle  
STOP
```

Pseudo Code 4: Advanced Lane Finding

```
START  
Read Road Image  
Correct Distortion  
Apply Threshold  
Detect Lane Pixels  
Fit Lane Curve  
Display Lane  
STOP
```

Pseudo Code 5: Vehicle Detection

```
START  
Capture Image  
Detect Vehicles  
Draw Bounding Boxes  
Display Vehicles  
STOP
```

Pseudo Code 6: Extended Kalman Filter

```
START  
Receive Radar/Lidar Data  
Predict Object State  
Update State Estimate  
Output Position  
STOP
```


Pseudo Code 7: Unscented Kalman Filter

```
START  
Generate Sigma Points  
Predict State  
Update Measurement  
Estimate Position  
STOP
```

Pseudo Code 8: Kidnapped Vehicle

```
START  
Initialize Particles  
Predict Particle Motion  
Update Particle Weights  
Resample Particles  
Estimate Location  
STOP
```

Pseudo Code 9: PID Control

```
START  
Calculate Error  
Compute PID Output  
Generate Steering Command  
Control Vehicle  
STOP
```

Pseudo Code 10: MPC Control

```
START  
Predict Future States  
Optimize Control Actions  
Generate Steering Output  
Control Vehicle  
STOP
```

Pseudo Code 11: Path Planning

```
START  
Read Vehicle Position  
Generate Candidate Paths  
Evaluate Paths  
Select Best Path  
Output Trajectory  
STOP
```

Pseudo Code 12: Road Segmentation

```
START  
Read Road Image  
Apply FCN Model  
Classify Pixels  
Generate Road Mask  
Display Result  
STOP
```

Pseudo Code 13: Traffic Light Classifier

```
START  
Capture Traffic Light Image  
Detect Traffic Light  
Classify Signal Color  
Generate Decision  
STOP
```

CHAPTER 7

CONCLUSION

The Self-Driving Car System developed in this project demonstrates the successful integration of various autonomous driving technologies into a unified framework. The system combines Computer Vision, Deep Learning, Sensor Fusion, Localization, Path Planning, and Vehicle Control techniques to enable autonomous navigation in a simulation environment.

The perception layer of the system effectively identifies lane markings, traffic signs, traffic lights, road boundaries, and surrounding vehicles using image processing and deep learning algorithms. Behavioural Cloning enables the vehicle to learn steering behaviour directly from human driving data, while road segmentation and object detection improve environmental awareness. These modules help the vehicle understand and interpret complex road scenarios accurately.

Localization and state estimation are achieved using Extended Kalman Filter (EKF), Unscented Kalman Filter (UKF), and Particle Filter techniques. These algorithms improve the accuracy of vehicle positioning and object tracking by reducing sensor noise and uncertainty.

The overall results demonstrate that the developed system can successfully perform autonomous driving tasks such as lane keeping, steering prediction, object tracking, and trajectory following. This project provides a strong foundation for future research in autonomous vehicles and highlights the significant role of Artificial Intelligence and Computer Vision in developing safe, intelligent, and efficient transportation systems.

CHAPTER 8

REFERENCES

- 1.Liu, Z., Cai, Y., Wang, H. et al. Surrounding Objects Detection and Tracking for Autonomous Driving Using LiDAR and Radar Fusion. Chin. J. Mech. Eng. 34, 117 (2021).
- 2.Alaa Aldeen Housein, Gao Xingyu, Weiming Li and Yang Huang. “Extended Kalman Filter Sensor Fusion in Practice for Mobile Robot Localization”. International Journal of Advanced Computer Science and Applications (IJACSA) 13.2 (2022).
- 3.Yanwei Li, Hengshuang Zhao, Xiaojuan Qi, Liwei Wang, Zeming Li, Jian Sun, Jiaya Jia. "Fully Convolutional Networks for Panoptic Segmentation" (2021)
- 4.Adam Paszke, Abhishek Chaurasia, Sangpil Kim, Eugenio Culurciello. "ENet: A Deep Neural Network Architecture for Real-Time Semantic Segmentation"
- 5.Alexander Wong, Mohammad Javad Shafiee, Michael St. Jules."MicronNet: A Highly Compact Deep Convolutional Neural Network Architecture for Real-time Embedded Traffic Sign Classification"
- 6.Sangam Prasad, Shankar Desai, Sandeep kumar, Adarsha M V, Dr. R.Guru, “Traffic Sign Detection Using CNN,” International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE),
- 7.Vaidya, B., Paunwala, C. (2020). Traffic Sign Recognition Using Color and Spatial Transformer Network on GPU Embedded Development Board. In: Nain, N., Vipparthi, S., Raman, B. (eds) Computer Vision and Image Processing. CVIP 2019. Communications in Computer and Information Science, vol 1147. Springer, Singapore

8.A. Dhawale, K. S. Shankar, and N. Michael, "Fast Monte-Carlo Localization on Aerial Vehicles using Approximate Continuous Belief Representations,"

9.N. Akai, T. Hirayama, and H. Murase, "3D Monte Carlo Localization with Efficient Distance Field Representation for Automated Driving in Dynamic Environments," 2020 IEEE Intelligent Vehicles Symposium (IV), pp. 1588-1595, 2020

10.P. Shi, L. Li, X. Ni, and A. Yang, "Intelligent vehicle path tracking control based on improved MPC and hybrid PID," IEEE Access, vol. 10, pp. 94133–94144, 2022,

11.M. Samuel, K. Yahya, H. Attar, A. Amer, M. Mohamed, and T. A. Badmos, "Evaluating the performance of fuzzy-PID control for lane recognition and lane-keeping in vehicle simulations," Electronics, vol. 12, no. 3, art. no. 724, 2023,